B2 — DPO and IPO on a language model: task, goal, and how it was solved

What phase B2 asks, why a preference loss can stand in for B1's RL loop, where that shortcut breaks, and what the runs showed. The lab notebook with the numbers is `experiments/b2_dpo/README.md`; every table and figure here is produced by `experiments/b2_dpo/report.py` from the run directories.

Contents

- 1. The task
- 2. The goal
- 3. What was done

1. The task

`ladder.md` states three items:

1. Build or synthesise a preference dataset; train DPO; compare against the best PPO run.
2. Overfit DPO on a small preference set; show chosen-logprob collapse and KL blowup.
3. Swap in IPO's bounded objective; compare stability.

2. The goal

B1 maximised the KL-regularised reward

$$\max_{\pi} \; \mathbb{E}_{x,\,y\sim\pi}\big[r(x,y)\big] - \beta\,\mathrm{KL}\big(\pi(\cdot\mid x)\,\|\,\pi_{\mathrm{ref}}(\cdot\mid x)\big)$$

with a sampling loop. This objective has a closed-form maximiser,

$$\pi^*(y\mid x) = \frac{1}{Z(x)}\,\pi_{\mathrm{ref}}(y\mid x)\,e^{r(x,y)/\beta}, \qquad \text{so} \qquad r(x,y) = \beta\log\frac{\pi^*(y\mid x)}{\pi_{\mathrm{ref}}(y\mid x)} + \beta\log Z(x).$$

Any reward is therefore a policy in disguise. If preferences follow the Bradley-Terry model, $P(y_w \succ y_l) = \sigma\big(r(x,y_w) - r(x,y_l)\big)$, the partition function cancels in the difference, and fitting the reward by maximum likelihood becomes fitting the policy directly. With

$$h_\theta = \log\frac{\pi_\theta(y_w\mid x)}{\pi_{\mathrm{ref}}(y_w\mid x)} - \log\frac{\pi_\theta(y_l\mid x)}{\pi_{\mathrm{ref}}(y_l\mid x)},$$

the losses are

$$\mathcal{L}_{\mathrm{DPO}} = -\log\sigma(\beta\,h_\theta), \qquad \mathcal{L}_{\mathrm{IPO}} = \Big(h_\theta - \frac{1}{2\tau}\Big)^2.$$

No reward model, no sampling, no value function: a classification loss over pairs, with the reference's log-probabilities computed once.

The shortcut has a cost the derivation hides. Bradley-Terry needs $P(y_w \succ y_l) < 1$ to give a finite reward difference. When a pair is labelled the same way every time, the maximum-likelihood h is infinite, and DPO's loss keeps falling as h grows *whatever β is*: the KL term that anchored B1's policy exists in the derivation, not in the loss being minimised. The loss also constrains only the *difference* of two log-ratios, so it can be lowered by pushing the rejected response down without lifting the chosen one. IPO (Azar et al., 2023) replaces the log-sigmoid with a squared distance to a finite target margin $1/(2\tau)$, so a pair that is already separated by that much stops pulling. Items 2-3 ask whether those failure modes show up, and whether IPO's bound actually removes them.

3. What was done

Task, policy, judges: B1's, unchanged. Continue the opening 4-8 tokens of an IMDB review with up to 48 sampled tokens, positively. Qwen2.5-0.5B (base) with LoRA (r = 16, all linear layers); the reference is the same model with the adapters off. Evaluation on B1's 64 held-out test-split openings, sampled with B1's fixed seed: B1's reward model score (the Qwen head PPO trained against), the gold judge's P(positive) (`siebert/sentiment-roberta-large-english`, never trained against), KL to the reference (sum over completion tokens of $\log \pi - \log \pi_{\text{ref}}$ on the policy's own samples), distinct-2, reference NLL and the share of samples containing Chinese (PPO's hack in B1).

The preference set (`make_prefs.py`). For each of 4,096 train-split openings, four continuations sampled from the reference policy at temperature 1, each scored by B1's reward model; the pair is the best- and worst-scored of the four (`--pair-rule random` takes two at random instead). 512 test-split openings give held-out pairs the same way. These are synthetic preferences from the same reward model PPO optimised, so the comparison with PPO is about the optimiser, not the labels: PPO saw the reward through 4,800 online rollouts, DPO through four fixed reference samples per prompt. Labels are deterministic (the higher score always wins), which is the regime the IPO paper argues DPO handles worst.

Training (`dpo_lm.py`, written out rather than through TRL so every log-probability is visible). Sequence log-probabilities summed over completion tokens; batches of 16 pairs; AdamW with 10 warm-up updates; fp16 autocast on Kaggle T4s. Logged every update on the training batch, and every 25 updates on 256 held-out pairs: loss, preference accuracy ($h > 0$), h, the chosen and rejected log-ratios to the reference, and the chosen and rejected log-probabilities per token.